

OSSP PRACTICAL WEEK-9

TASK-1:

Implement a file copy utility using low-level Linux file I/O system calls such as `open()`, `read()`, `write()`, `lseek()` and `close()`. Compare its performance with standard library functions.

SOURCE CODE:

```

#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>
#include <time.h>

#define BUFFER_SIZE 4096

double get_time()
{
    struct timespec ts;
    clock_gettime(CLOCK_MONOTONIC, &ts);
    return ts.tv_sec + ts.tv_nsec / 1e9;
}

/* Low-level system call copy */
void low_level_copy(const char *source, const char *destination)
{
    int src, dest;
    char buffer[BUFFER_SIZE];
    ssize_t bytes_read, bytes_written;

    src = open(source, O_RDONLY);
    if (src < 0)
    {
        perror("open source");
        exit(1);
    }

    dest = open(destination, O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (dest < 0)
    {
        perror("open destination");
        close(src);
        exit(1);
    }
}

```

```

    exit(1);
}

dest = open(destination, O_WRONLY | O_CREAT | O_TRUNC, 0644);
if (dest < 0)
{
    perror("open destination");
    close(src);
    exit(1);
}

while ((bytes_read = read(src, buffer, BUFFER_SIZE)) > 0)
{
    bytes_written = write(dest, buffer, bytes_read);

    if (bytes_written != bytes_read)
    {
        perror("write");
        close(src);
        close(dest);
        exit(1);
    }
}

close(src);
close(dest);
}

/* Standard library copy */
void standard_copy(const char *source, const char *destination)
{
    FILE *src, *dest;
    char buffer[BUFFER_SIZE];
    size_t bytes_read;

```

```
/* Standard library copy */
void standard_copy(const char *source, const char *destination)
{
    FILE *src, *dest;
    char buffer[BUFFER_SIZE];
    size_t bytes_read;

    src = fopen(source, "rb");
    if (src == NULL)
    {
        perror("fopen source");
        exit(1);
    }

    dest = fopen(destination, "wb");
    if (dest == NULL)
    {
        perror("fopen destination");
        fclose(src);
        exit(1);
    }

    while ((bytes_read = fread(buffer, 1, BUFFER_SIZE, src)) > 0)
    {
        fwrite(buffer, 1, bytes_read, dest);
    }

    fclose(src);
    fclose(dest);
}

int main()
```

```
int main()
{
    double start, end;

    printf("File Copy Performance Comparison\n");
    printf("-----\n");

    start = get_time();
    low_level_copy("input.txt", "lowlevel.txt");
    end = get_time();

    printf("Low-level I/O copy completed.\n");
    printf("Time using open/read/write/close: %.6f seconds\n",
           end - start);

    start = get_time();
    standard_copy("input.txt", "standard.txt");
    end = get_time();

    printf("Standard library copy completed.\n");
    printf("Time using fopen/fread/fwrite/fclose: %.6f seconds\n",
           end - start);

    printf("\nBoth files copied successfully.\n");

    return 0;
}
```

Output:

```
sairam0806@LAPTOP-M6NQUIAP:/mnt/c/Users/SAIRAM0806$ nano filecopy.c
sairam0806@LAPTOP-M6NQUIAP:/mnt/c/Users/SAIRAM0806$ gcc filecopy.c -o filecopy
sairam0806@LAPTOP-M6NQUIAP:/mnt/c/Users/SAIRAM0806$ cat input.txt
This is a sample file used for testing file copy performance.
sairam0806@LAPTOP-M6NQUIAP:/mnt/c/Users/SAIRAM0806$ ./filecopy
File Copy Performance Comparison
-----
Low-level I/O copy completed.
Time using open/read/write/close: 0.007561 seconds
Standard library copy completed.
Time using fopen/fread/fwrite/fclose: 0.008330 seconds

Both files copied successfully.
sairam0806@LAPTOP-M6NQUIAP:/mnt/c/Users/SAIRAM0806$ |
```

OBSERAVTION:

The source file was successfully copied to the destination file using low-level Linux file I/O system calls. The contents of both files were verified and found to be the same.

TASK 2 :

Implement a file copy utility using low-level Linux file I/O system calls such as `open()`, `read()`, `write()`, `lseek()` and `close()`. Compare its performance with standard library functions.

SOURCE CODE :

```
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>
#include <time.h>

#define BUFFER_SIZE 4096

double get_time()
{
    struct timespec ts;

    clock_gettime(CLOCK_MONOTONIC, &ts);

    return ts.tv_sec + ts.tv_nsec / 1000000000.0;
}

void low_level_copy()
{
    int src, dest;
    char buffer[BUFFER_SIZE];
    ssize_t bytes;

    src = open("input.txt", O_RDONLY);

    if (src == -1)
    {
        perror("open");
        exit(1);
    }

    dest = open("lowlevel.txt",
               O_WRONLY | O_CREAT | O_TRUNC, 0644);

    if (dest == -1)
    {
        perror("open");
    }
}
```

```
{
    perror("open");
    close(src);
    exit(1);
}

while ((bytes = read(src, buffer, BUFFER_SIZE)) > 0)
{
    write(dest, buffer, bytes);
}

close(src);
close(dest);
}

void standard_copy()
{
    FILE *src, *dest;
    char buffer[BUFFER_SIZE];
    size_t bytes;

    src = fopen("input.txt", "rb");
    dest = fopen("standard.txt", "wb");

    if (src == NULL || dest == NULL)
    {
        perror("fopen");
        exit(1);
    }

    while ((bytes = fread(buffer, 1, BUFFER_SIZE, src)) > 0)
    {
        fwrite(buffer, 1, bytes, dest);
    }

    fclose(src);
    fclose(dest);
}
```

```

    fclose(src);
    fclose(dest);
}

int main()
{
    double start, end;
    int fd;
    off_t size;

    printf("FILE COPY PERFORMANCE COMPARISON\n");
    printf("-----\n");

    start = get_time();

    low_level_copy();

    end = get_time();

    printf("Low-level copy completed.\n");
    printf("Time using open/read/write/close: %.6f seconds\n",
        end - start);

    start = get_time();

    standard_copy();

    end = get_time();

    printf("Standard library copy completed.\n");
    printf("Time using fopen/fread/fwrite/fclose: %.6f seconds\n",
        end - start);

    fd = open("input.txt", O_RDONLY);

    if (fd != -1)

```

```

fd = open("input.txt", O_RDONLY);

if (fd != -1)
{
    size = lseek(fd, 0, SEEK_END);

    printf("File size using lseek(): %ld bytes\n",
           (long)size);

    close(fd);
}

printf("\nBoth files copied successfully.\n");

return 0;
}
|

```

Output:

```

sairam0806@LAPTOP-M6NQIUAP:/mnt/c/Users/SAIRAM0806$ ./filecopy
FILE COPY PERFORMANCE COMPARISON
-----
Low-level copy completed.
Time using open/read/write/close: 0.010529 seconds
Standard library copy completed.
Time using fopen/fread/fwrite/fclose: 0.011898 seconds
File size using lseek(): 62 bytes

Both files copied successfully.

```

OBSERVATION : The source file was successfully copied to the destination file using lowlevel Linux file I/O system calls. The contents of both files were verified and found to be the same.

